Machine Learning-Based Intrusion Detection System for Detecting SQL Injection Attacks in Web Applications

Alqaiyas Qasim Al Sulaimi

This thesis is presented in partial fulfilment the requirements of the bachelor's degree in
Computer Science
In
Cyber Security

Department Computer Science
College of Engineering and Computer Sciences
German University of Technology in Oman
Sultanate of Oman
June 2026

Student's Name: Alqaiyas Qasim Al Sulaimi (21-0027)

Title of Thesis: Machine Learning-Based Intrusion Detection System for Detecting SQL Injection Attacks in Web Applications

Thesis approval date 26/Feb/2026 Thursday

Thesis Committee:

1.
Supervisor Dr. Waseem Raja Anwar
Title Assistant Professor
Department Computer Science
Institution German University of Technology in Oman

2.
Co-Supervisor Mr. Ali Al Humairi
Title Senior Lecturer
Department Computer Science
Institution German University of Technology in Oman

Statutory Declaration

I, Alqaiyas Al Sulaimi, hereby declare that I have written this thesis in all parts by myself and have not used any sources and aids other than those specified in the thesis, and that the thesis has not been presented in the same or a similar form in any other examination. Any part designed by AI-tools or sections that were edited or proofread using AI-based tools are identified.

I further declare that I shall not publish, disseminate, reproduce, or make publicly available this thesis, in whole or in part, in any form or medium, without the prior written consent of my main supervisor.

Sultanate of Oman, Muscat signature
24/05/26

Acknowledgment

First and foremost, I extend my deepest gratitude to my thesis supervisors, Dr. Waseem and Dr. Nabil, for their unwavering guidance, support, and invaluable insights throughout this research. Their expertise and encouragement have been instrumental in the successful completion of this project. I also wish to acknowledge my own perseverance and commitment in overcoming the challenges encountered during this research journey.

I am sincerely grateful to my family and friends for their constant support, patience, and encouragement. Their belief in my abilities has been a continuous source of motivation.

I would also like to thank my colleagues for sharing their knowledge and providing moral support during difficult times. Finally, I extend my appreciation to all who contributed, directly or indirectly, to this research, including the Information Security team at Dhofar Insurance Company, whose cooperation and assistance were invaluable during my training and throughout the thesis process.

Alqaiyas Al Sulaimi

Abstract

The widespread number of SQL injection attacks represents one of the most critical threats to web applications, demanding development of efficient and reliable detection techniques. This thesis proposes SQLInsight, a state-of-the-art framework to detect SQLi attacks in real-time by leveraging machine learning, particularly the logistic regression classifier, to enhance web application security. The proposed model has been trained by making use of a pre-processed dataset to optimize the accuracy and reduce the number of false positives. Operating on the server-side, SQLInsight can be integrated seamlessly with any web infrastructure as it not only alerts web administrators in real-time via email notification upon successful identification of any malicious inputs but also provides a real-time monitoring dashboard allowing administrators an intuitive visualization of any attack trends. It offers a wide variety of data concerning threat level as well as the status of web application security to the web administrators, which will help in taking appropriate measure toward counter any impending attack on their web application. This system combining machine learning, real-time alerting and an interactive dashboard makes for a powerful and practical tool for Web Security and aims to provide a robust, scalable and easily implementable method for detection of SQL Injection attacks. Future work can involve implementing a mechanism for detecting various type of web attacks and incorporating this system into existing web security infrastructure.

Keywords: SQL Injection Attack, Web Application Security, Logistic Regression, Cybersecurity, Machine Learning, Intrusion Detection System, Real-Time Detection, Monitoring Dashboard

TABLE OF CONTENTS

Statutory Declaration ii

Acknowledgment iii

Abstract iv

Table of Contents v

List of Figures vi

List of Tables vii

List of Equations viii

Chapter 1: Introduction 13

1.1 General Introduction 13

1.2 Problem Statement 14

1.3 Research Questions 15

1.4 Research Objectives 15

1.5 Research Scope 15

Chapter 2: Background 16

2.1 What does SQL Injection Mean? 16

2.2 SQL Injection Mechanism 17

2.3 SQL Attack Types 18

2.4 Detection Techniques 20

2.4.1 Log Analysis 20

2.4.2 Honeypots 20

2.4.3 Intrusion Detection System 21

2.5 Common Machine Learning Algorithms in IDS 22

2.5.1 Machine Learning Models 22

Chapter 3: Literature Review **24**

3.1 Related Works 24

3.1.1 SQL Injection Detection Using Traditional Methods 24

3.1.2 Machine Learning 25

3.1.3 Neural Network 27

3.2 Findings 27

Chapter 4: Design Structure and Methodology **28**

4.1 Development Approach 28

4.2 System Architecture 29

Chapter 5: Implementation **32**

5.1 Environment Setup 32

5.1.1 Hardware and Software Requirements 32

5.2 Website and Server Setup 33

5.3 Data Collection 34

5.4 Import Libraries 35

5.5 Data Preprocessing 35

5.5.1 Preprocessing Queries 35

5.6 Training and Testing 37

5.7 Evaluation Criteria 38

5.8 Deployment 40

Chapter 6: Results and Discussion **41**

6.1 First Experiment 41

6.2 Second Experiment 43

6.3 Comparison with Previous Approaches 45

6.4 Integration and Deployment in Real-World Application 46

6.5 Challenges and Limitations 47

Chapter 7: Conclusion **48**

7.1 Future Works 49

Glossary **50**

Bibliography **53**

Appendix A: ML Model Training Code **56**

Appendix B: Server-side Programming Code **59**

LIST OF FIGURES

Figure 1 SQL Injection Methodology 17

Figure 2 Honeypot System 20

Figure 3 Machine Learning Models 22

Figure 4 SQLInsight Structure 29

Figure 5 Website's Main Page 33

Figure 6 Search Bar 33

Figure 7 Script of Importing Necessary Libraries 35

Figure 8 Dataset Before Processing 35

Figure 9 Script of Cleaning and Preprocessing the Dataset 35

Figure 10 Script of Training and Testing the Model 37

Figure 11 Performance Evaluating the Trained Model 38

Figure 12 Script of Saving and Downloading Trained Model and Vectorizer 40

Figure 13 Performance Metric (First Experiment) 42

Figure 14 Script of Loading and Testing the New Dataset 43

Figure 15 Performance Metric of New Dataset 45

Figure 16 Real-World Integration and Alert System 46

LIST OF TABLES

Table 1 Software Prerequisites 32
Table 2 Hardware Prerequisites 32
Table 3 Dataset Summary Overview 34
Table 4 Division of the Dataset 37
Table 5 Confusion Matrix 39
Table 6 Results of First Experiment 41
Table 7 Results of Second Experiment 44
Table 8 Comparison between Previous Research and Current Study's Findings 46

LIST OF EQUATIONS

Equation 1 Logarithmic Function (Logistic Regression Sigmoid) 23
Equation 2 Accuracy 38
Equation 3 Precision 38
Equation 4 Recall 38
Equation 5 F1 Score 38

Chapter 1: Introduction

The chapter sets the stage for the research, which focuses on the development of an automated detection system against SQL injection attacks and malicious scripts. It introduces the significance of cybersecurity in today's digital life, especially in web application security. The chapter identifies the research questions, describes the purposes, and describes the typical SQL injection attack methods, underlining the necessity and emphasising the need for robust detection mechanisms.

1.1 General Introduction

There has been a lot of advancement in different aspects of modern technology and information technologies, because this has seen a radical shift in our day-to-day lives and has broken the shackles that were delaying the process of growth and development, which has made life easier to many humans and enabled them to conduct their day-to-day activities easily and comfortably. Nevertheless, despite the numerous advantages of technological advancement, there is a huge dark side that brings security issues, and cybersecurity is a source of great concern in the digital era. Organizations and individuals face an increasing range of threats that aim to penetrate sensitive information and data and cause huge losses to victims. As these cyber-attacks have become more complicated and have various objectives and aims, immediate measures need to be taken to minimize the potential risks and mitigate their severity.

Security has become a very crucial issue in the development of web applications due to the sheer quantity of information housed on the internet and the increase in sharing of data between people and companies. They are susceptible to several attacks including illegal activities in different industries and daily transactions. Hence, the priority of ensuring that this data is not exploited in any way and is safeguarded remains the main aim to further technological progress.

Structured Query Language (SQL) injection attacks have become one of the most dangerous cyber threats, attacking web applications and databases maliciously with the aim of exploitation. Weaknesses in web programming or improper setup allow attackers to access sites unlawfully, enabling the reading of stored information and data. One of the sites regarded to be most exposed to the dangers of SQL injection attacks are WordPress sites, as they are very prolific in the world and offer few barriers of defence.

To overcome these shortcomings, this study suggests creating proactive cybersecurity solutions to improve the security posture of web applications through the use of Machine Learning (ML) techniques, advanced analysis of web traffic patterns, and a real-time threat detection mechanism. The proposed project will help address the associated risks, guard against this kind of attack, and prevent unauthorized access to personal data. Furthermore, to enhance the usability and effectiveness of the system, a real-time monitoring dashboard will be developed to provide administrators with a clear and interactive visual interface to monitor detected threats, track attack patterns, and respond promptly to security incidents.

There are a number of methods that an intruder can employ in this type of attack to gain unauthorized access to databases and extract information. This enables the attacker to access sensitive information of individuals and organizations; thus, the hacker can easily alter or erase this data, resulting in loss of data, damaged reputation, loss of money, and alterations in the behaviour of the application. The primary techniques used by the attacker to execute the attack are categorized into four types, including: injection through cookies, user input injection, server variable injection, and second order or stored injection.

In terms of data retrieval, these SQL Injection (SQLi) attacks can be classified into three major types: In-band SQLi, Out-of-band SQLi, and Inferential SQLi.

This introduction paves the way for scientific research, highlights the importance of cybersecurity in the current era, and emphasizes the importance of taking preventive measures to protect against cyberattacks to reduce material or moral losses. In this study, an approach will be proposed known as **SQL Insight** to identify SQL injection attacks, provide real-time warnings, and present detected threats through an interactive monitoring dashboard, based on the use of the logistic regression (LR) approach.

1.1 Problem statement

Based on the OWASP list of the 10 most common cyber-attacks of 2017–2021 [1], SQL injection is still the most prevalent type of attack on websites by attackers to use websites to their benefit, with unauthorized access to databases. Such attacks may cause data breach, manipulation of data, and other security threats to organizations, which emphasizes the need to establish effective strategies to solve the problem of SQL injection attacks and improve the security stance of web applications and databases. **SQLInsight** strategy uses the logistic regression (LR) for real-time threat detection, which is intended to improve the web application security and defence against SQL injection attacks.

1.2 Research Questions

The key questions of this study are to respond to the following critical questions:

- What are the existing SQL injection detection methods weaknesses, and how does the suggested machine learning solution overcome them?
- What can an SQL injection detector based on machine learning effectively be integrated to a live web environment?
- What is the effectiveness of the logistic regression model at identifying SQL injection attacks as compared to traditional methods?
- What are the impacts of SQL injection detection system implementation on the security posture of a web application?
- What can be done to make SQL injection attack detection more robust to changing attack strategies?

1.3 Research Objectives

The key aims of the study are the following:

- To examine and confirm the use of logistic regression as an effective tool in identifying SQL injection attacks via model training using normal and malicious SQL payloads.
- To monitor the effectiveness of the deployed system in a real-world scenario and its performance.
- To compare the performance of **SQLInsight** systematically with the existing SQL injection detection methods, emphasizing accuracy improvements, data size, and computation efficiency.
- To determine possible areas of future research and development.

1.4 Research Scope

The focus of this project is strictly limited to the identification of SQL injection attacks through a logistic regression model. It only aims at detecting and notifying administrators of SQLi attempts in real time via emails and live monitor as well, without going so far as to prevent or recover such attacks. Additionally, this detection mechanism is applied only to the server side, and it scans and inspects the incoming SQL inquiries that do not deal with activities or interactions on the client side.

Chapter 2: Background

The background chapter of this thesis report gives a detailed description of common SQL injection attack detection techniques with their strengths and weaknesses. Furthermore, the chapter introduces machine learning (ML) and its various types, emphasizing their potential for improving SQL injection detection. This preconditions the following discussion of the application of ML in SQL injection attack detection on the thesis project.

2.1 what does mean SQL Injection?

SQL injection attacks were thought to be one of the most famous and well-known web attack means which is implemented by hackers to steal data from web applications. This method can allow hackers to play around with database queries, which may have a huge impact on hacking confidential information, modifying documents, or corrupting data. SQL injection attacks should be a significant cyber threat that results in data breaches, economic impacts, and organizational reputational losses. A countermeasure to these risks can be input validation, parameterized queries, and secure coding practices. It is essential to educate people and implement strong security controls to effectively fight SQL injection attacks and improve database security against abuse [2].

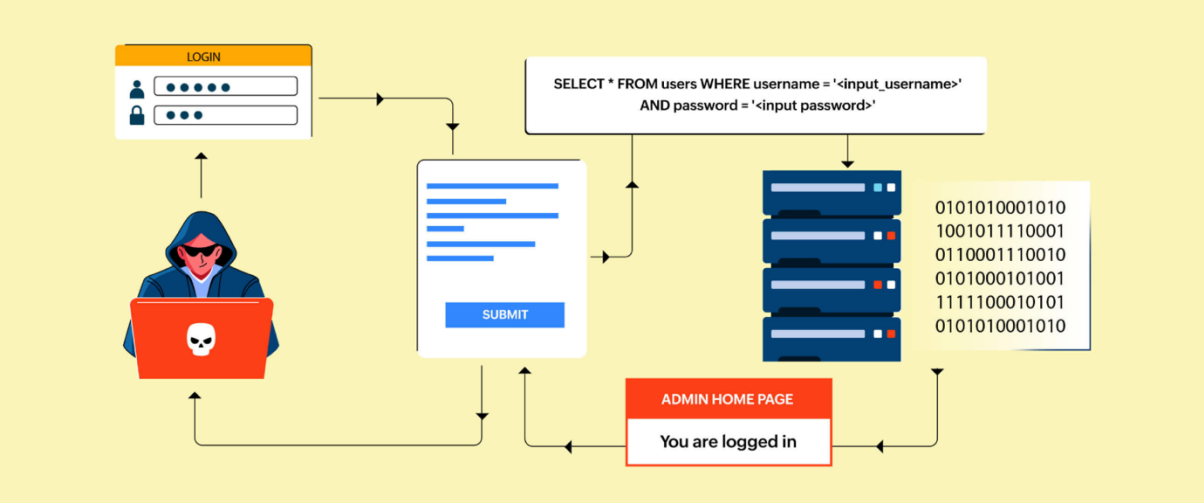

Figure 1: SQL Injection methodology [3]

2.2 SQL Injection Mechanism

Any parameter used in a database query that is part of an application can be used as an SQL injection vulnerability, allowing the start of an SQL injection attack. There are various methods an attacker can use to breach databases. These methods have been called injection mechanisms and are of four major types:

- **Injection via cookies:** Cookies are mechanisms that guarantee continuity to web applications by saving state information on the user's device. In order to resume a session, the cookie is often retrieved by the user through the web application. SQL injection by cookies is a common weakness of web applications, where malicious users can use cookies to inject malicious SQL code into a web application. To illustrate, a hacker might alter a cookie value to insert a SQL injection payload, which would be passed on to the application at the time the cookie is processed. This form of injection attack can circumvent security controls and manipulate database records [4].

- **User input-based injection:** This form of SQL injection is the most common. Attackers can compromise user input fields such as login forms and search bars with malicious SQL code. As a case in point, an attacker might enter a SQL injection code into a login form to circumnavigate authentication and access a system unauthenticated [5].

- **Injection with server variables:** Server variables are the parameters of the network describing the Hyper Text Transfer Protocol (HTTP) request and environment. This includes the two most popular ways of submitting forms, GET and POST, as well as manipulating HTTP headers. These variables may be manipulated by the intruder to provide malicious SQL code. For example, an attacker may alter the HTTP header to comprise SQL commands. If the server-side code combines these variables into SQL queries, it can lead to SQL injection vulnerabilities [5].

- **Second order injection:** Second order injection, also known as stored injection, is where the malicious SQL code is not executed immediately. Rather, it is saved in the database and run later. As an example, a hacker might insert harmful SQL code into a comment box on a web page, which is stored in the database. When the comment is later recalled and displayed on the site, the malicious SQL code is executed, resulting in a SQL injection attack. This form of injection is especially perilous since the attack is not initiated immediately but is triggered by the system itself at a later point [6].

2.3 SQL Attack Types

This section explains the key categories of SQL injection attacks, such as In-band, Out-of-band, and Blind SQL injections, also referred to as inferential. The importance of learning these types of attacks is to create the best understanding of preventive and mitigation measures.

The easiest type of SQL injection is in-band attacks, which entail retrieving the results via the same channel used to inject SQL code. The information obtained when in-band injection attacks occur is shown directly on the application webpage.

- **Union based SQLi:** The SQL UNION operator is used to join the results of multiple select queries within one set of results. Attackers may generate malicious queries that put extra data into the result of the original query, so that they can derive sensitive information from the database. Attackers can access data that was not originally meant to be published by manipulating the structure of the SQL query, posing a substantial security risk to the application [7].

  As an example, the query that is run on the server is the following: `SELECT first_name, last_name FROM users UNION SELECT username, password FROM login;` The SELECT query will select the first name and last name from the users table. Then the results of a second SELECT statement are appended using the UNION operator, which retrieves the columns of username and password from the login table. The query is successful when it consists of the correct number of columns [7].

- **Error based SQLi:** In this type, the attacker exploits error messages generated by a database to test the SQL queries by purposely triggering them to generate errors, thereby obtaining information about its structure. This allows attackers to steal table data and even sensitive user data. This type of attack depends on error-handling mechanisms in the database to leak out information that can otherwise be exploited in the future [7].

I**nferential SQLi**, also called Blind SQLi attack, enables attackers to deduce the data in a database system despite the system being secure enough to not expose any false information to the attacker, by using conditional techniques and timing attacks.

- **Blind SQLi:** In this category, the attacks are independent of error messages or visual responses of the database, and not on information returned by SQL queries, which makes them more challenging and threatening. Hackers make designed queries to the database and draw conclusions according to the performance of the application. Either using delay-to-answer time or logic-based attacks, the attackers are able to retrieve information from the database without directly viewing the database output. This kind of attack needs a more advanced method of detecting and preventing it, since it can avoid traditional security measures [7].

Out-of-band SQLi attack is not very common, and it is based on the existence of certain features being turned on in the database server used by the web application.

- **Out-of-Band SQLi:** These are rare SQL injection attacks since they are based on alternative communication channels that are not in the database, such as Domain Name System (DNS) scraping or HTTP requests, which are required to retrieve data from the database. Attackers bypass conventional data retrieval techniques, which alone create a special challenge to defenders, requiring expertise and mechanisms to detect these malicious activities [7].

2.4 Detection Technique

It is crucial to have proper controls to identify SQL injection attacks for both businesses and people, because these attacks can be very costly. The next section will discuss popular detection techniques, such as log analysis, intrusion detection systems, and honeypots.

2.4.1 Log Analysis

Log file analysis is a key procedure in computer security and may be employed to indicate signs of malicious activities such as unauthorized access, or to uncover security vulnerabilities in a system. Both the web server and the database server produce log files to record data about the requests that are made to the web application. One way to utilize log analysis for identifying SQL injection attacks is by checking the log files to determine whether a SQL query was being executed with malicious code. Logs can be evaluated either by hand or by computer. One of the elements of manual analysis is the line-by-line examination of log files to detect SQL injection attacks. Log files can also be analysed using automated systems in real time, which notify when there are indicators of suspicious patterns, and can be used to improve security. Log analysis can fully monitor and trace all requests across the system. This significantly makes it easier to detect and correct security vulnerabilities, but the main disadvantage is that anomaly detection can be time consuming and difficult to compute quickly and efficiently with large and heterogeneous log data [8], [9].

2.4.2 Honeypots

A honeypot is a trap system that is meant to attract and trap possible attackers. Honeypots are special in presenting themselves as defenceless systems to attract attackers, with the intention of deceiving them and learning their techniques in order to improve security. Essentially, honeypots are built to be studied, monitored, and hacked, to be used as detection and response mechanisms rather than preventive measures. Honeypots can be used to identify SQL injection attacks as they act as a vulnerable web application to attract hackers' attention, enabling the honeypot to serve as an attack target. If an attacker can insert malicious code into the spoofed web application, the honeypot will identify the assault and provide evidence of it. Since honeypots enable an attacker to attempt to inject malicious code into a SQL query in a controlled environment, they are very effective at detecting SQL injection attempts. Because of this, the honeypot can successfully recognize an intrusion attempt and is less likely to cause false positives than other detection systems, rendering them an attractive choice. Nonetheless, improper management of honeypots may cause malicious actors to infiltrate the actual system. The primary disadvantage of honeypots is probably the enormous effort it requires to set up and maintain them [8], [10]. Figure 2 below demonstrates that a honeypot may be a part of the network.

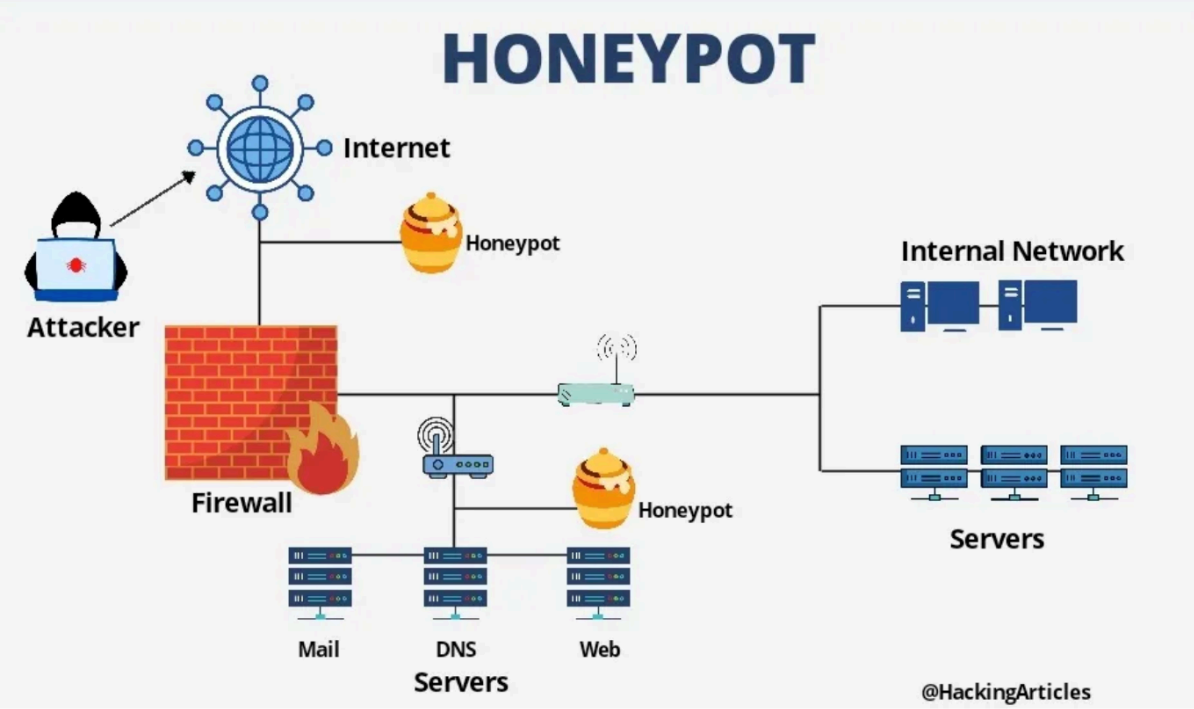

Figure 2 : Honeypot System [11]

2.4.3 Intrusion Detection System

A combination of programs and systems is referred to as intrusion detection systems (IDS), used to identify cyber-attacks or possible intrusions which pose risks to the integrity of the system, such as SQL injection attacks. IDS is a watchdog that is vigilant and keeps an eye on network traffic or host activities to detect any malicious activity or policy breach. The first intrusion detection system was suggested in 1980, and since then numerous models with new technologies have emerged [12]. Data source-based intrusion detection systems are of two main types: host intrusion detection systems (HIDS) and network intrusion detection systems (NIDS).

NIDS monitors and scans traffic in networks and analyses any kind of incoming and outgoing packet of a network, whereas HIDS is dedicated to the investigation of host-level events. IDS relies on various detection techniques in order to detect possible threats, such as signature-based, behaviour-based, and anomaly-based methods. Signature-based detection performs a comparison between network traffic and a database, seeking signatures that have already been detected in the database. In behaviour-based detection, system activity and processes are monitored to detect any abnormal, possibly malicious behaviour. An anomaly-based detection system, on the other hand, looks for unknown suspicious behaviour that does not conform to the baseline, including attempting to log in during non-business hours. Although there are numerous advantages that IDS offers, there are certain challenges, such as the difficulty of separating real threats from false positives [13], [14].

2.5 Common Machine Learning Algorithms in IDS

The incorporation of machine learning algorithms in Intrusion Detection Systems (IDS) has significantly enhanced their ability to detect and respond to a wide array of cyber threats. Based on the trends and behaviours of network traffic and system logs, machine learning can be leveraged to enable IDS to detect anomalies and possible intrusions more precisely and effectively. This section discusses some of the most popular machine learning algorithms in IDS, pointing out their functions in enhancing threat detection and the overall strength of security measures.

2.5.1 Machine Learning Models

Machine Learning (ML) is one of the significant improvements in the sphere of Artificial Intelligence (AI), because it enables systems to automatically learn and improve through experience. This technology paves the way to create and automate computer programs to handle large datasets by improving the self-learning functions of machines without human involvement, thereby performing tasks more dependably and swiftly. The learning process starts by giving the model a great diversity of information and data along with instructions, in order to search for patterns in the data to make better decisions in the future. Machine learning is mostly applicable in circumstances where one needs to analyze a large dataset and identify patterns that humans may not be able to discern [15].

This boom has transformed numerous industries, including the healthcare industry, whereby it aids in diagnosing diseases, to the commercial world, where it forecasts the trend of the economy and the stock market. As machine learning algorithms keep advancing, the reliability and ease of use of these technologies will enhance their adoption and incorporation into different aspects of life, providing the opportunity to propel development. There are two fundamental types of machine learning: supervised learning and unsupervised learning. Supervised learning is based on labelled data to learn and is used to train and construct models using this labelled data. On the contrary, unsupervised learning algorithms are based on unlabelled data, extracting specific information and discovering complex relationships without instructions or directions. In Figure 3, the common machine learning algorithms used in IDS are shown [12].

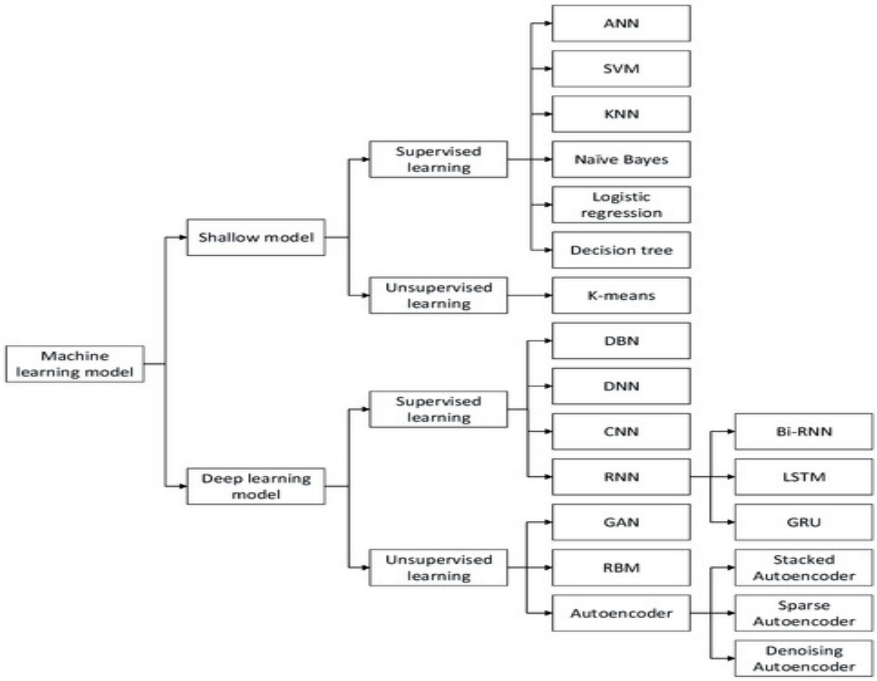

Figure 3 : Machine Learning Models [12]

2.5.1.1 Shallow Models

Shallow model learning, also called traditional ML, is a subdivision of machine learning algorithms that can generate generalized predictive models with a small number of layers of composition and is based on relatively simple models having a small number of processing stages. These models are suitable when working with datasets that have well-defined discriminative features. In the context of Intrusion Detection Systems (IDS), shallow models are important. These models include Artificial Neural Networks (ANN), Support Vector Machines (SVM), K-Nearest Neighbor (KNN), Naive Bayes, Logistic Regression (LR), decision trees, clustering, and other hybrid combination methods. Although their emphasis is on detection effectiveness, these models also cover more practical issues, as they form the basis of the cybersecurity field by providing detection efficiency. Shallow models are

beneficial because of their uncomplicated design, ease of training, and simplicity of interpretation. However, they may have limitations in capturing complex patterns and generalizing to new data, which are crucial considerations in the context of IDS. This paper focuses on the use of Logistic Regression (LR) as an effective machine learning system to counter malicious payloads amongst normal payloads [16], [17].

$$P\,(Y = k|x) = \frac{e^{w_k * x}}{1 + \sum_{k}^{K-1} e^{w_k * x}}$$

Equation 1 Logarithmic function

Logistic Regression (LR) is a machine learning classification algorithm that is classified as a type of supervised learning technique. It classifies data into separate categories by learning the relationships between the classified dataset [18]. It is especially specialised in forecasting binary results, such as 'Yes' or 'No', 0 or 1, and True or False, because it involves the use of the logistic function as an activation of real-valued numbers to probabilities in [0,1]. Integrating logistic regression into intrusion detection techniques helps to improve performance efficiency and the capability to attain good outcomes [19]. In detecting SQL injection attacks, it is necessary to identify whether a web request is malicious or normal. Therefore, the use of logistic regression is considered useful and highly accurate in the forecast of such cases. Logistic regression is characterized by its simplicity, interpretability, and performance in binary classification. Nevertheless, LR is not able to deal with non-linear data, which limits its wider applicability [12]. The following equation (Equation 1) calculates the probability of various classes with a parametric logistic distribution of the LR algorithm.

2.5.1.2 Deep Learning Model

Deep Learning is a type of machine learning and artificial intelligence based on neural networks. It is a strong algorithm inspired by brain architecture, enabling computers to learn complicated ideas through practice without the need to be explicitly taught. Deep learning models include a wide range of deep networks, such as Deep Belief Networks (DBNs), Deep Neural Networks (DNNs), Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), autoencoders, Restricted Boltzmann Machines (RBMs), and Generative Adversarial Networks (GANs) [20]. One of the most important benefits of deep learning is that it can directly learn feature representations based on raw data, such as images and texts, without the need for manual feature engineering. This allows deep learning to simplify and improve the work process. Additionally, deep learning techniques are distinguished by their high ability to deal with large datasets in terms of performance strength and efficiency, compared to shallow models which are not capable of these. This has contributed to a great breakthrough in various fields, such as Artificial Intelligence, Image Processing, Robotics, and Automation [12].

Chapter 3: Literature Review

The third chapter conducts a comprehensive review of relevant literature, including an exploration of available literature on cybersecurity, with an emphasis on conceptualizing the concepts and detection processes associated with this cybersecurity threat.

3.1 Related Works

In this part, a brief review of the relevant literature is given, including the spectrum of SQL injections and corresponding measures to detect and prevent them. This comprehensive review would help to provide clarity on the current research environment and developments in both detection and prevention mechanisms, enabling a more in-depth knowledge of how to effectively protect against SQL injections.

3.1.1 SQL Injection Detection Using Traditional

Conventional SQL injection detection techniques are based on established methods such as blacklist filtering, whitelist validation, string matching algorithms, penetration testing, and static analysis frameworks. Blacklist filtering blocks illegal characters, while whitelist validation supports only safe, predefined characters. String matching algorithms are used to match URL strings with known injected SQL strings to determine possible threats. Penetration testing is a simulation of attacks aimed at discovering vulnerabilities, and static analysis models such as SAFELI can be used to identify SQL code injection vulnerabilities [21].

Xiang Fu and Kai Qian proposed the SAFELI [22] SQL injection vulnerability detector using symbolic execution to analyse Java web applications' bytecode. SAFELI constructs equations on the web control inputs to determine possible attacks and creates test cases to re-run SQL injection attacks among programmers. The main contributions of the paper are the combination of symbolic execution tools and bytecode, which give a more detailed analysis to detect vulnerabilities that might be hard to identify with conventional security vulnerability scanning devices on the black box. The study, however, recognizes gaps such as the limited functionalities of the framework in non-Microsoft environments.

Abdullayev and Chauhan provide a detailed discussion of the prevention and detection methods of SQL injection attacks [8], which address the dire and continual threat these attacks pose to web applications. One of the basic defence mechanisms is input validation, to ensure that user input is carefully vetted to avoid the injection of malicious code into SQL queries. Parameterized queries are also mentioned as a powerful approach to increasing security by isolating user-supplied data from SQL code, making it more challenging for attackers to exploit vulnerabilities. However, the review does not include case studies or real-world applications, which would benefit the applicability of the results in practice.

The paper [23] outlines an innovative technique that employs signature-based analysis to perform detection based on the behaviour of malware. The method is specifically built to detect the actions of known malware and to reveal any kinds of malicious activities that

might escape detection from traditional methods of static analysis. These are useful in combating advanced malware that may use obfuscation or polymorphism to avoid being detected by conventional means. Nevertheless, this method has some significant drawbacks, including its reliance on existing malware signatures, which can minimize its resistance to new or unknown variants. Moreover, the dependence on behavioural analysis can allow more false positives, particularly in dynamic settings where legitimate activities may be mistaken for malicious behaviours.

A study by Zeli Xiao, Zhiguo Zhou, and Weiwei Yang provides a novel approach for detecting SQL injection attacks by using a dynamic detection system based on behaviour analysis and response analysis. This two-way method improves the accuracy of detection and minimizes false positives in comparison with conventional single-layer algorithms. Such a system operates by scraping URL addresses and access log files of the web server and transforming them into five sets of signatures to check and verify the k-folder. The undesirable nature of this framework is its dependence on pre-structured basic behavioural baselines, which might not match new forms of SQL injection techniques [24].

3.1.2 Machine Learning

The article [8] delves into the importance of proactive measures to identify and thwart SQL injection attacks. Log analysis is introduced as a useful tool to analyze web server and database logs to identify indications of malicious code being executed in SQL queries. Intrusion Detection Systems (IDS) are also mentioned as the necessary means of real-time monitoring of network traffic to detect security threats such as SQL injections. The review highlights the importance of integrating preventative and detection techniques to strengthen the security stance of organizations against changing threats, and the necessity of having a holistic approach to security that combines several ways of reducing risks.

Machine learning approaches have been popularized in SQL injection detection based on data-driven strategies to determine patterns which signal attacks. These methods involve deriving features from URL access logs and using machine learning to identify attempts of SQL injection. Algorithms such as Support Vector Machines (SVM), random forest, and Naive Bayes have been used to automatically categorize web pages and detect malicious URLs. Models like the Cuckoo Search SVM model have been constructed to derive features and build hybrid classifiers to identify phishing emails. Machine learning offers an adaptable and dynamic solution to SQL injection detection, which allows systems to develop and enhance their detecting abilities over time [21].

A survey by Tareek Pattewar et al. is a detailed study on the use of Naive Bayes and Gradient Boosting machine learning algorithms in detecting and defending against SQL injection in dynamic web applications. The proposed solution supports the use of supervised learning methods to categorize SQL queries as either benign or malicious, with the goal of improving the accuracy and effectiveness of threat detection. The methodology described in the study entails training machine learning models on datasets that include SQL queries, feature extraction to obtain the right characteristics, and the implementation of classification algorithms to distinguish between legitimate and possibly harmful queries. By incorporating machine learning processes into cybersecurity, the study strives to proactively mitigate SQL

injection risks and automate the process of uncovering vulnerabilities, so as to strengthen the security measures of web applications against new cyber attacks [25].

Moh et al. conducted a study that delves into a multi-stage log analysis approach as a detection mechanism for web attacks [26], and SQL injection attacks in particular. The proposed solution combines pattern matching and supervised machine learning techniques to improve detection capabilities. The methodology applies offline training with WEKA and Bayes Net, including preprocessing techniques such as stemmers, removal of stop words, and cross-validation to optimize the training process. The implementation involves a proof-of-concept prototype running on Amazon AWS, with Kibana for pattern matching and a Bayes Net for machine learning. While the study emphasizes the merits of the multi-stage log analysis method, it does not exclude the drawbacks, such as the necessity to constantly update to meet changing methods of attack and the difficulty of effectively managing large volumes of log information. Future research directions may seek to examine ways to scale and automate the updating process and improve scalability for real-time detection in high-traffic web environments.

The authors present a dynamic prevention strategy intended to increase the safety of web applications against SQL injection attacks. The technique revolves around dynamically comparing the defined query structures, which are specified by programmers, and the actual query formed at runtime, to proactively detect and prevent SQL injection vulnerabilities. Using parse tree validation techniques, the method focuses on automatically detecting unauthorized SQL query modifications, thus enhancing the security stance of web applications. This dynamic prevention approach aligns with the principles of active security by being proactive in countering possible vulnerabilities without manual intervention. The application of this method demonstrates its efficiency and ease of integration, which underscores its effectiveness in preventing SQL injections and protecting web applications against malicious exploits [27].

3.1.3 Neural Network

Multi-Layer Perceptron (MLP) and Long Short-Term Memory (LSTM) are sophisticated devices for SQL injection detection. These networks resemble the interconnectedness of neurons in the human brain, which allows them to identify complex patterns in large datasets. Deep learning algorithms have proven to be effective at identifying web security threats. Scientists have developed deep learning structures to identify malicious JavaScript code, fingerprint attacks on websites, and subspace spectral ensemble clustering to detect web attacks. With the power of neural networks, organizations can bolster their ability to detect and thwart SQL injection attacks effectively [28].

A detailed SQL injection prevention system based on the use of neural networks was presented in research paper [21]. The system structure entailed a study of normal URLs and SQL injection URLs using big data analysis to find certain features that relate to SQL injection attacks. By extracting these URL features, the neural network was trained on a binary classification task with the aim of distinguishing between normal and malicious URLs. The trained model was then deployed on an Internet Service Provider (ISP) big data platform to perform real-time SQL injection detection against user HTTP traffic data. This

implementation led to a robust SQL injection detector which relies on neural networks, using the metadata supplied by the ISP as input to analyse and extract features to train the model. With the trained model in place in the ISP system, it was possible to immediately identify abnormal network behaviours, enhancing overall cybersecurity measures.

Potluri et al. provide an in-depth discussion on Intrusion Detection Systems (IDS) [29], by adding Convolutional Neural Networks (CNN) to enhance the accuracy of detection of different types of attacks on Industrial Control Systems (ICS). CNN networks are popular due to their ability to process two-dimensional data, in which network packets are transformed into images for CNN processing. The gap, however, lies in the insufficient investigation of new attacks, which requires frequent retraining on the training datasets.

The method of insider threat identification proposed by colleagues is more advanced [30], as it uses deep learning methods. The model's ability to detect anomalies in real time and analyse them is one of its principal strengths, allowing security analysts to comprehend how these features are used and how specific attacks occur. The authors demonstrate that their models are more effective than traditional methods such as Principal Component Analysis (PCA), SVM, and Isolation Forests, especially in achieving high recall rates for threat detection. Nevertheless, the use of the Computer Emergency Response Team (CERT) Insider Threat Dataset v6.2 raises concerns that the conclusions might not generalize well to different environments.

3.2 Findings

The analysis of the related works points out some major results and emphasizes the originality of the approach of **SQLInsight** in dealing with SQL injection (SQLi) threats. Old techniques of SQL injection detection, such as blacklist filtering and string-matching algorithms, have been widely used but frequently tend to have large false positive rates and are not very good at identifying new attack patterns. Deep learning and machine learning methods have more sophisticated features to detect SQLi attacks through the analysis of trends in big data. Nevertheless, deep learning models are computationally demanding and require large amounts of data for training. Conversely, machine learning algorithms such as logistic regression (LR) offer a trade-off between high accuracy and reduced computational overhead. LR was chosen for **SQLInsight** for its ability to efficiently classify SQL queries as malicious or benign. It is a viable and convenient predictor for binary classification and thus a popular and efficient selection for SQLi detection in real-time.

Chapter 4: Design Structure and Methodology

The methodology section of the study describes the systematic approach that was used in the attainment of the study, to define the research objectives and answer the research questions. Methodological rigour is needed to ensure the validity, reliability, and generalizability of study results. This part outlines the study design, data collection procedures, data analysis procedures, and all other procedures performed to conduct the research.

4.1 Development Approach

An automated monitoring system needs to be developed through a systematic approach to avert SQL injection attacks. It will involve, but not be limited to, the following:

1. **Literature Review:** The methodology began by conducting research and reviewing the available literature on the detection and protection techniques against SQL injection attacks on web applications. This review not only provided an accurate picture of the situation at hand, but also indicated the gaps in the current responses, which require further research to compensate and make up the deficit.

2. **System Design:** Based on the recommendations of the research literature and lessons learned, the system will be designed. The lessons learnt in the literature review include the fact that logistic regression is simple and efficient in binary classification, and hence it will be chosen as the machine learning model.

3. **Implementation:** Python language will be employed to design the system to simplify implementation, and because it is machine learning friendly. This is the step that will be devoted to training and developing the detection model on the basis of a diversity of data that will be collected from different source environments.

4. **Testing:** Intensive tests will be conducted to establish the level of performance of the system, to ensure that it is effective in the detection of SQL injection attacks. Testing will include the quantification of model performance measures such as accuracy and precision.

4.2 System Architecture

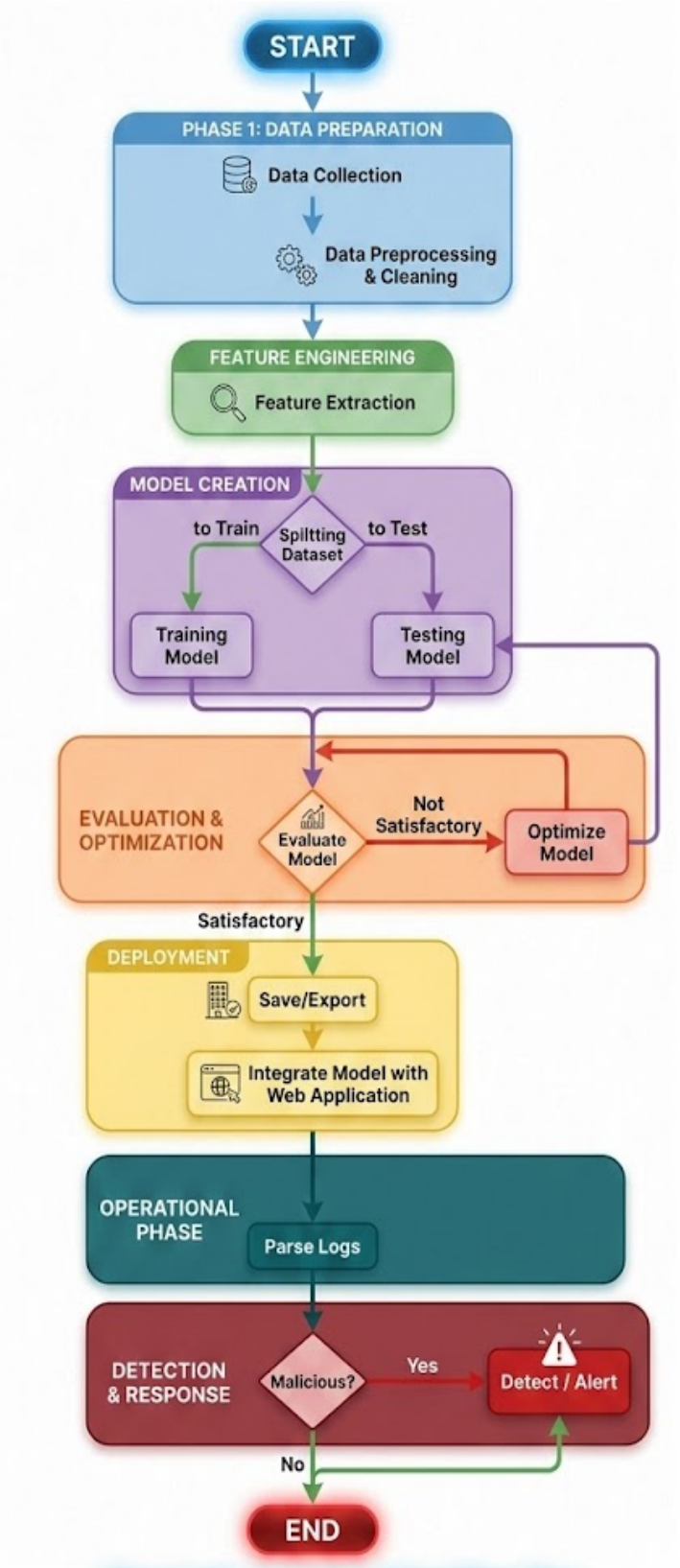

Figure 4 SQLInsight Structure

The proposed framework of detecting SQL injection attacks in a web application consists of two layers. The user layer contains the first layer, which is used to send user requests. The second layer represents the protection layer and includes the proposed scheme of classifying the requests received by the first layer. The layer consists of many phases as shown below:

Stage 1: Data Collection

Diverse sets of data samples such as malicious and benign are collected and combined to be utilized in the machine learning model training and evaluation. Some of these datasets include web traffic logs, databases, and other relevant information with regards to the HTTP requests and database interactions. The data is marked as 0 (Valid SQL query) and 1 (Malicious SQL query).

Stage 2: Cleaning and Pre-Processing.

The data obtained are subjected to a cleaning and pre-processing stage in the process of eliminating any non-relevant data and standardizing the data format. This includes data cleaning, transformation and integration methods to get the data ready to analyze.

Stage 3: Feature Extraction

The preprocessed data is extracted to extract relevant features that reflect the important characteristics of the HTTP requests and database interactions. Such features can consist of HTTP request headers, URLs, SQL queries, single or double quotes, use of SQL keywords, and other signs of possibly malicious activity.

Stage 4: Training and Testing the Dataset.

The dataset is separated into training and testing datasets with a traditional proportion of 80-20 (80 percent training and 20 percent testing). This split helps to formulate and test the machine learning model, make it strong, and generalized.

Stage 5: Evaluate Model

Various performance metrics are used to evaluate the trained machine learning model, such as accuracy, precision, recall, and F1-score. This step evaluates how well the model works in separating between normal and malicious HTTP requests. In case the results of the evaluation are not satisfactory, the model is optimized and re-evaluated.

Stage 6: Save/Export

After training and testing the model, it is stored or exported to a file to be used again. This enables easy implementation as well as integration with the back-end infrastructure of the web application.

Stage 7: Integrate Web Application with Model.

The trained machine learning model is deployed as a part of the web application backend so that it can be used to monitor incoming HTTP requests in real-time in case of a possible SQL injection attack. This integration will provide proactive security threat detection and mitigation.

Stage 8: Parse Logs

Incoming HTTP requests are analyzed to get appropriate information, which includes request methods, URLs, and query strings. This processed data is fed into the machine learning model to be analysed, which can effectively identify suspicious activity.

Stage 9: Detect/Alert

The machine learning model will examine the received HTTP requests after parsing them to identify any indication of SQL injection attacks. In case a suspicious request is detected, an alarm system is activated to alert the owner or administrator of the site so that immediate response can be taken to avert the risk.

Stage 11: Dashboard of Real-Time monitoring.

An interactive visual interface is created and incorporated in the system by developing a real-time monitoring dashboard. The dashboard shows important security data including the total amount of detected SQL injection attacks, dates of suspicious requests, attack frequency patterns, log of system alerts, etc. This will allow administrators to keep track of the security level of the web application and address arising threats in a timely manner.

Chapter 5: Implementation

This chapter covers the implementation of SQLInsight, an SQL injection detection tool for Web apps. It includes the selected approach, needs, coding procedure, data collection methods and data analysis methods. The chapter explains how scripts and algorithms were created to process the access logs of web servers and identify SQL injection attacks, and how a real-time monitoring dashboard was built to provide administrators with a visual representation of any threats that are detected. It offers a complete picture of the way in which the concepts and approaches of previous chapters are applied to a working system, and helps to move from theory to practice by demonstrating the implementation of the developed strategy.

5.1 Environment setup

5.1.1 Hardware and Software Requirements

The student will describe the hardware and software needed to support the creation of digital media projects. Any system development, especially one using ML, should have a set of basic hardware and software specifications. In this study, Tables 1 and 2 provide a listing of the essential prerequisites used for creating **SQLInsight**. These requirements were the basis of the development, deployment, and assessment of the system's impact.

Software requirement	
System Type	macOS 64-bit
Programming language	Python 3.12

Table 1: Software prerequisites

Hardware Requirements	
Processor	Intel Core i7 (Quad-Core)
Processor Speed	1.2 GHz
RAM	16 GB 3733 MHz LPDDR4X
Hard Disk	256 GB SSD
Graphics	Intel Iris Plus Graphics (1536 MB)

Table 2: Hardware prerequisites

5.2 Website and Server Setup

The first step was to get an IP address of the domain to allow for the proof of concept of the SQL injection detection system. After that, a WordPress website was developed that has a search field to mimic user interaction and potential attack vectors. The website has been designed to be user friendly such that the user can search the website using a search query to provide a realistic environment in which to test the detection capabilities of the logistic regression model when it encounters SQL injection attacks. The website was then set-up on an Apache server, which is a secure web server that can be accessed from the internet. This involved configuring the server to capture all real-time queries of all searches with the tail -f command. This setup was important in watching user behaviour and spotting SQL injection attempts. The environment was also configured to support the server, with the real-time logging mechanism playing a crucial role in effectively monitoring and analysing search queries. Furthermore, a real-time monitoring dashboard was added to the system to give administrators a visual interface to see the SQL injection attacks that are being detected, to look at the alert logs, and to see what attacks are occurring in real-time.

Figure 5 website's main page

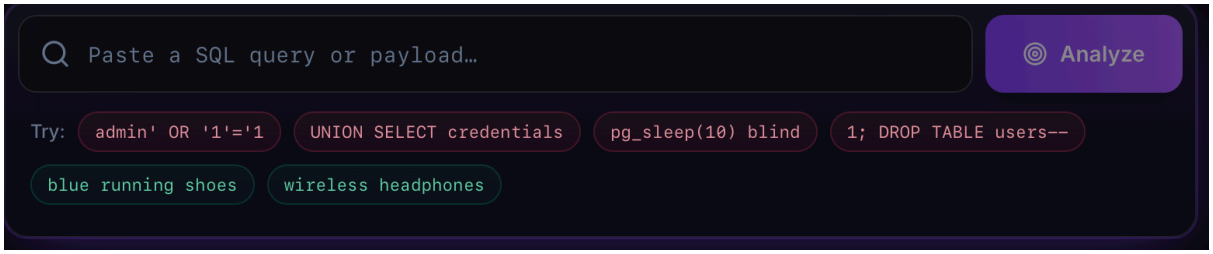

Figure 6 Search Bar

5.3 Data Collection

Machine learning algorithms need to be trained with data relating to the research problem to be used for training and testing of the model. The data used in this research were collected from various open-source sources including the Kaggle website [31] and GitHub [32] and totalled 148,326 samples, including malicious and benign payloads. The data set contains one SQL statement per entry and its label. The dataset is organized in the following way:

Query: This is the actual SQL query strings. These queries contain queries that are commonly used for database operations and malicious queries trying to exploit vulnerabilities in the database system.

Label: Labelled as 1 if query is malicious, and 0 if it is a normal query.

Name of Dataset	Number of samples	Normal	Malicious
SQL_Dataset	148,326	70,576	77,750

Table 3 Dataset summary overview

5.4 Import Libraries

The codebase (Figure 7) starts with importing the necessary dependencies for data analysis and machine learning. Initializes the environment for a machine learning project, preprocessing data, and imports tools to use Google Colab, create models, and evaluate them.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import drive
from sklearn.feature_selection import VarianceThreshold
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score,f1_score
from sklearn.metrics import confusion_matrix
import joblib
import shutil
from google.colab import files
```

Figure 7 Script of Importing Necessary Libraries

5.5 Data Preprocessing

It is important to preprocess the data to make it more comprehensible to the machine learning models and to fit it correctly and remove redundant values and outliers. It is an important step for creating any model. The main goal of this approach is to make data preparation easier.

Query
" or pg_sleep (__TIME__) --
create user name identified by pass123 temporary tablespace temp default tablespace users;
AND 1 = utl_inaddr.get_host_address ((SELECT DISTINCT (table_name) FROM (SELECT DISTINCT (table_name) , ROWNUM AS LIMIT FROM sys.all_tab
select * from users where id = '1' or @ @1 = 1 union select 1,version ( ) -- 1'
select * from users where id = 1 or 1#" (union select 1,version ( ) -- 1
select name from syscolumns where id = (select id from sysobjects where name = tablename') --
select * from users where id = 1 +$+ or 1 = 1 -- 1
1; (load_file (char (47,101,116,99,47,112,97,115,115,119,100))) ,1,1,1;
select * from users where id = '1' or ||/1 = 1 union select 1,version ( ) -- 1'
select * from users where id = '1' or \.<\ union select 1,@@VERSION -- 1'
? or 1 = 1 --

Figure 8 Dataset Before Processing

5.5.1 Preprocessing Queries

The process of preparing data includes: minimizing the amount of data, data integration, standardizing data, removing anomalies or duplicate values, and removing empty values. First, the text was transformed into a list of words by CountVectorizer without altering the case of original text. Next, a VarianceThreshold filter was used to drop terms that had low variance, which should filter out both too common and too rare terms. All the words were then lowercased using CountVectorizer to standardize the data. In the end, missing values were dropped in the data to create a clean dataset. The pre-processing steps were essential in normalizing the text data and making it amenable to training the ML model that will identify SQL injection attacks. It is shown that the code for data preprocessing is as shown in the following figure (Figure 9).

```
# Stage 1: Convert text into a collection of words
vectorizer = CountVectorizer(lowercase=False)
X = vectorizer.fit_transform(data['Query'])
# Stage 2: Remove frequently and infrequently used terms
sel = VarianceThreshold(threshold=(.0001 * (1 - .0001)))
X = sel.fit_transform(X)
# Stage 3: Convert every word into lowercase
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(data['Query'].str.lower())
y = data['Label']
# Stage 4: Remove missing rows
data.dropna(inplace=True)
```

Figure 9Script of Cleaning and Preprocessing the Dataset

The code starts with the line "with open(csv_filename) as f:" which opens the CSV file to read the dataset, which includes the features needed to train and test the model. The text in the ‘Query' column is converted to a matrix of token counts using the CountVectorizer from scikit-learn.The text in the ‘Query' column is represented as a matrix of token counts using the CountVectorizer from scikit-learn. Words which occur only once in the document are considered as unique words and the number of each word in a document is used as its value in the matrix. In this part of the process the parameter lowercase=False means that the words shouldn't be turned to lowercase, so that the case of the words stays the same.

In the next stage a VarianceThreshold feature selector is applied to the matrix of tokens X obtained in the previous step. This feature selector extracts features (words) with minimal variance among all the samples. In this case, features that have little variance will be features that are common or rare in the data set. These terms can be eliminated, giving a smaller, more efficient and effective model.

Another CountVectorizer instance is created, without the lowercase attribute, so the text is converted to lowercase in this phase. This will make sure that the text is recorded in the same case, which will help to standardize the data of the text words and make the feature space simpler. Normalizing case of words may also assist the performance of some text processing tasks, like text classification.

Finally, the dropna function allows to eliminate those rows from the dataset where there are missing values. Missing values can be problematic for machine learning models, as they can lead to errors or biases in the analysis. This helps to eliminate noise and prepare the data for subsequent analysis or processing.

5.6 Training and Testing

In the third step of an ML approach, the data is split into two different sets by calling the train_test_split function. This function shuffles data randomly and divides the data into two parts: X_train, y_train, which is used for training, and X_test, y_test, which is used for testing. The test_size parameter is set to 0.2, which means that 20% of the data will be used for testing and 80% for training. The split is reproducible with the help of the random_state=42 argument.

The dataset is split into two parts, and then a Logistic Regression model is fitted with the help of LogisticRegression from scikit-learn. This model is then fit to the training set (X_train, y_train) with the fit method. The max_iter parameter defines the number of iterations that the optimization algorithm will run; the solver parameter defines the optimization algorithm to use. In training, the model is given the relationship between input features (X_train) and the target labels (y_train) and then tries to make a prediction for new data (X_test) that it has never seen before.

```
# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Creating and Training the Model
model = LogisticRegression(max_iter=1000, solver='liblinear') # liblinear is mo
model.fit(X_train, y_train)
# Making predictions on the test set
y_pred = model.predict(X_test)
# Evaluating the Model
```

Figure 10 Script of Training and Testing the model

Table 4 is the Summary of the SQL_dataset table showing the number of samples and their corresponding "SQL ID", Distribution of training and testing sets used to develop and evaluate the models.

Name of Dataset	Number of samples	Training Mode	Testing mode
SQL_dataset	148,326	118,660	29,666

Table 4 Division of the Dataset

5.7 Evaluation Criteria

In the last phase of the prediction approach, a number of performance parameters are computed to assess the model's performance and to choose the best model. These metrics are accuracy, precision, recall and f1 score. The trained Logistic Regression model is applied to the test set (X_test) in this code snippet (Figure 11) to predict the target labels for the test set.

```
# Evaluating the Model
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
print(f"Precision: {precision*100:.2f}%")
print(f"Recall: {recall*100:.2f}%")
print(f"F1 Score: {f1*100:.2f}%")
print(classification_report(y_test, y_pred))
```

Figure 11 Performance Evaluating the Trained Model

The following are the measurements which will be used for evaluation of the algorithmic method, in addition to its efficiency:

Accuracy (A) is the fraction of correct classifications to the total number of samples.

$$A = \frac{\mathrm{TP+TN}}{\mathrm{TP+FP+FN+TN}}$$

Equation 2 Accuracy

Precision (P) refers to the proportion of accurate positive cases in relation to all the identified positive cases, which indicates how sure one can be about detecting the attacks.

$$P = \frac{TP}{TP+FP}$$

Equation 3 Precision

Recall (R) = True positives / Total positives, This metric evaluates the model's efficiency in detecting attacks.

$$R = \frac{TP}{TP+FN}$$

Equation 4 Recall

F1 Score (F) is calculated as the harmonic mean of precision and recall.

$$F = \frac{2*P*R}{P+R}$$

Equation 5 F1 Score

Different values have been used within the confusion matrix to obtain these measures. As shown in Table 5, the confusion matrix has the following classification types: False Positive (FP), False Negative (FN), True Negative (TN), and True Positive (TP).

	Class X	Class Y
Class X	TP	FP
Class Y	FN	TN

Table 5 Confusion Matrix

- TP: It is used to refer to the payloads which harm that have been predicted accurately by the model.

- TN: This stands for cases which have been predicted by the prediction approach to be benign and indeed they are.

- FN: This stands for cases in which prediction approach misclassifies the harmless case as harmful.

- FP: It stands for cases in which prediction approach misclassifies malicious case as benign.

5.8 Deployment

After training, the machine learning algorithm is deployed as part of the website in order to detect SQL injection attacks on a continuous basis. The steps taken during deployment include ensuring the deployment of the trained Logistic Regression model into the server. Firstly, the trained Logistic Regression model is saved to enable it to be deployed later on. Secondly, the logistic regression model is deployed in the web server where it will be used to detect any suspicious activities.

For the purpose of detecting suspicious SQL injection attacks, a python script is written and used to continually analyze data in the access log files through the use of the tail –f command. This analysis involves obtaining log entries from the web server's access log and preprocessing the data to get the query part of the entry. The query parts of log entries obtained are then analyzed using the Logistic Regression model and upon detecting a suspicious activity, an alert message is sent through e-mail to system administrators.

```
# Save the model
joblib.dump(model, 'ML_model.pkl')
# Define path
colab_model_path = 'ML_model.pkl'
# Download the model file
files.download(colab_model_path)
# Save the vectorizer
joblib.dump(vectorizer, 'vectorizer-2.joblib')
# Define path
colab_vectorizer_path = 'vectorizer-2.joblib'
# Download the vectorizer file
files.download(colab_vectorizer_path)
```

Figure 12 Script of Saving and Downloading trained model and vectorizer

The code shown above (Figure 12) shows the steps involved in saving and then downloading the trained machine learning model and the vectorizer. Using the `joblib.dump` command, the trained model (model) is serialized into a file labeled 'ML_model.pkl' while the vectorizer (vectorizer) is serialized into 'vectorizer-2.joblib.' These files save the state of both the model and the vectorizer, allowing for reuse without the need to train again. These two files have their paths saved in variables labeled as `colab_model_path` and `colab_vectorizer_path`, which are then utilized in the `files.download` command to download the two files from Google Colab to a local computer.

Chapter 6: Results and Discussion

In this chapter, the results of implementing the machine learning model in SQL injection detection will be provided. The efficiency of the model will be analyzed in accordance with several criteria including the model's accuracy, precision, recall, and F1 score. Besides, the practical implementation of the proposed solution will be presented together with its comparison with other solutions. Overall, the results of the conducted research can be considered as promising for the future development of the solution.

6.1 First Experiment

The first experiment involved a dataset with malicious and normal payloads. The dataset was divided into two parts where 80% of it were used for training and the other 20% for testing. As can be seen in Table 6, the system under consideration has obtained a high level of accuracy 99.08%.

From the experimental results, it is possible to state that the precision rate is equal to 99.03%, while the recall rate is 98.47%. Besides, F1 score equals 98.75%.

Index	Name of Parameter	Value
1	Model Accuracy	99.08%
2	Model Precision	99.03%
3	Model Recall	98.47%
4	Model F-Score	98.75%
5	Training Data Size	24,736
6	Testing Data Size	6,184

Table 6 Results of First Experiment

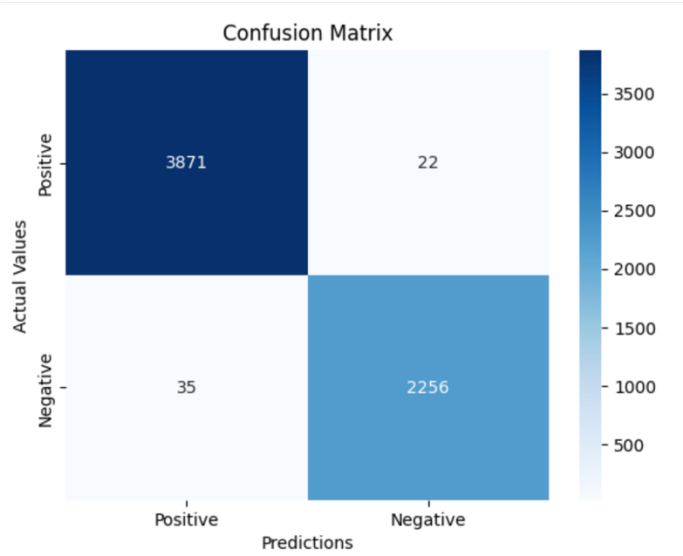

Figure 13 Performance Metric

From the confusion matrix presented in Figure 13, it can be noted that SQLInsight shows great success in the detection of SQL injection attacks. This method of intrusion detection shows high efficiency due to a high detection rate. It has 3,871 cases of true positives and 2,256 cases of true negatives. At the same time, the number of false positives is quite low and equals 35 cases, whereas the number of false negatives is 22 cases.

Thus, it can be stated that, together with data preprocessing and selection, the Logistic Regression model successfully detects SQL injection attacks with few errors.

6.2 Second Experiment

To further verify the efficiency and robustness of SQLInsight, a new unseen dataset was downloaded from Kaggle [33]. This dataset, comprising 70,576 positive cases and 77,750 negative cases, was used to evaluate the model's performance. The primary objective of this evaluation was to ensure that the model maintains strong detection capability and reliability when exposed to previously unseen data. The code snippet (Figure 14) demonstrates the preprocessing, prediction, and evaluation procedures.

The preprocessing stage involved transforming the queries in the new dataset using the same vectorizer applied during the training phase. Afterward, the trained model predicted the labels for the new dataset, and several evaluation metrics were computed. The experimental results achieved an accuracy of 84.51%, a precision of 79.04%, a recall of 95.88%, and an F1-score of 86.65%. Furthermore, the confusion matrix indicates that the model successfully identified 50,812 positive cases and 74,544 negative cases, while misclassifying only a limited number of samples.

These results demonstrate that SQLInsight generalizes effectively to unseen datasets and maintains a high recall rate, which is particularly important in cybersecurity applications where detecting malicious queries is critical. Although the precision is slightly lower due to some false positives, the overall performance confirms the practical applicability and robustness of the proposed model in real-world scenarios.

```
# Load the new dataset
new_data_path = '/content/drive/My Drive/SQLi/New_SQL_dataset.csv'
new_data = pd.read_csv(new_data_path)
# Preprocessing to the new dataset
new_X = vectorizer.transform(new_data['Query'])
new_y = new_data['Label']
# Predict using the trained model
new_predictions = model.predict(new_X)
# Evaluate the model on the new dataset
new_accuracy = accuracy_score(new_y, new_predictions)
new_precision = precision_score(new_y, new_predictions)
new_recall = recall_score(new_y, new_predictions)
new_f1 = f1_score(new_y, new_predictions)
print(f"New Data_Accuracy: {new_accuracy*100:.2f}%")
print(f"New Data_Precision: {new_precision*100:.2f}%")
print(f"New Data_Recall: {new_recall*100:.2f}%")
print(f"New Data_F1 Score: {new_f1*100:.2f}%")
# Confusion Matrix for the new dataset
new_conf_matrix = confusion_matrix(new_y, new_predictions)
sns.heatmap(new_conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=['Positive', 'Negative'], yticklabels=['Positive', 'Negative'])
plt.xlabel('Predictions')
plt.ylabel('Actual Values')
plt.title('Confusion Matrix for New Dataset')
plt.show()
```

Figure 14 Script of Loading and Testing the New Dataset

The dataset used in the second experiment included 148,326 instances consisting of both positive and negative SQL query samples, which were used to evaluate the performance of the proposed model on unseen data. The model achieved an accuracy of 84.51%, a precision of 79.04%, a recall of 95.88%, and an F1-score of 86.65%. These results indicate that the model maintains strong performance in detecting malicious SQL queries, particularly due to its high recall rate, which demonstrates its effectiveness in identifying most attack attempts. Although the precision value is comparatively lower because of some false positive predictions, the overall evaluation confirms that SQLInsight is capable of generalizing effectively to new datasets. Table 7 further demonstrates the effectiveness of SQLInsight in detecting SQL injection attacks, highlighting its suitability for real-world cybersecurity applications.

Index	Name of Parameter	Value
1	Model Accuracy	84.51%
2	Model Precision	79.04%
3	Model Recall	95.88%
4	Model F-Score	86.65%
5	Training Data Size	NA
6	Testing Data Size	148,326

Table 7 Results of Second Experiment

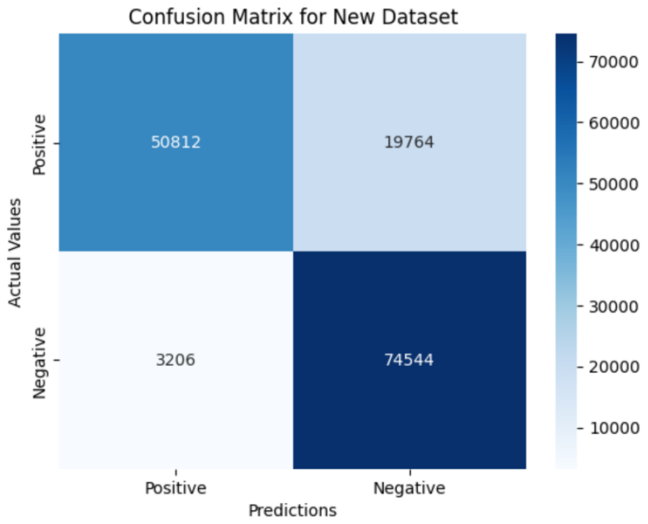

Figure 15 Performance Metric of New Dataset

Based on the confusion matrix presented in Figure 15, it can be stated that the SQLInsight model performs very effectively when it comes to detecting SQL injection attacks on a newly received dataset. The model correctly predicted 50,812 positive cases and 74,544 negative cases; however, at the same time, it also produced 19,764 false positives and 3,206 false negatives.

The following table presents the obtained evaluation results in terms of accuracy, precision, recall, and F1-score: Overall Accuracy = 84.51% Precision = 79.04% Recall = 95.88% F1-score = 86.65% It is clear that despite the fact that the recall rate turned out to be relatively high, this is quite an adequate result for such applications since high recall means that most of the malicious queries will be detected. Moreover, although the false positive rate is significantly higher than the false negative one, it does not prevent SQLInsight from making effective distinctions.

6.3 Comparison with Previous Approaches in Terms of Accuracy

As was revealed in the initial training and testing, SQLInsight showed great accuracy of SQL injection attack detection, which proved good training and generalization processes on the part of the model. In order to assess the practical value and reliability of the introduced model in terms of its application in real life, additional testing on the new unseen dataset was done. As a result, the model had an accuracy of 84.51% on the testing set, with precision of 79.04%, recall of 95.88% and F1-score of 86.65%.

Even though the accuracy received during additional testing was not as high as in previous experiments, it still showed very high recall rate, which proves the effectiveness of malicious code detection. In the field of cybersecurity, higher recall rates are more desirable than very high precisions, since any attacks should be detected and stopped, rather than let pass. As follows, SQLInsight demonstrates high capability to generalize different datasets other than the one used for training.

The verification of the proposed model using the new dataset becomes critical since this test determines how adaptable and robust the proposed method is in terms of new data distribution. Indeed, this study proves that SQLInsight still successfully detects any SQL injections, even when the number of queries increases, and their nature becomes more complex. Table 8 contains the comparison of existing research with the results obtained in this paper.

Ref	Model	Accuracy %	Data size
[1]	Neural network of direct signal propagation	95	30,233
[2]	Probabilistic neural network	95	4,511
[3]	Linear discriminant analysis	73.21	33,761
[4]	Naive Bayes classifier SQLInsight (First Experiment) SQLInsight (Second Experiment)	93.3 96.6 84.51	20,474 148,326 148,326

Table 8 Comparison between previous research and the current study's findings

6.4 Integration and Deployment in Real-World Application

The SQL injection detection model, SQLInsight, was successfully integrated and deployed into the backend of a real-world web application to evaluate its effectiveness in a live operational environment. As illustrated in (Figure 16), the system was configured to continuously monitor and analyze incoming web requests for potential SQL injection attacks in real time.

When a suspicious or malicious query was detected, SQLInsight immediately generated an automated security alert. The alert system provided essential information related to the detected attack, including the detection timestamp, source IP address, geographical location, attack status, and the complete log details of the malicious request. This real-time monitoring capability enables system administrators to respond quickly to potential threats and enhance the overall security of the web application.

The deployment results demonstrate that SQLInsight can operate effectively in practical environments while maintaining reliable SQL injection detection performance. Furthermore, the successful integration of the model into a live backend infrastructure confirms its applicability as a real-world cybersecurity solution for protecting web applications against SQL injection attacks.

6.5 Challenges and Limitations

Implementing the **SQLInsight** system involved several challenges and limitations. One significant challenge was ensuring the accuracy of the model while minimizing false positives and false negatives, as overly aggressive detection could block legitimate traffic, while lax detection could allow malicious activity. The variability in SQL injection techniques and the evolving nature of cyber threats also posed difficulties in maintaining a consistently high detection rate. Additionally, integrating the detection system into the existing website infrastructure required careful coordination to avoid disruptions to normal operations. Furthermore, the system faced challenges due to the lack of available datasets. The existing datasets were insufficient or not up-to-date, which affected **SQLInsight**'s ability to effectively recognize and adapt to emerging SQL injection patterns. Moreover, the inaccuracy of existing data due to a poor filtering process deleted entries containing data, which further limited the model's effectiveness.

Chapter 7: Conclusion

As SQL injection threats to web applications become more common, a mechanism to detect them is necessary in order to lower potential risks and enhance protection. This research has been able to create the SQLInsight application to identify SQL injection attack attempts, employing machine learning with the use of logistic regression. It shows that SQLInsight can provide a very high rate of accuracy in recognizing the SQL attacks and distinguishing them from benign ones in an efficient manner because of its vast and varied training data set.

The significance of this mechanism is the inclusion of real-time email alerts as well as real-time dashboards for monitoring to improve the effectiveness of SQLInsight in such a way that the administrator can quickly respond to the problem by undertaking necessary measures. It is because the dashboard serves as an interface through which the trends of attacks can be easily monitored and analyzed by the administrator. In this manner, not only will the security of web applications be improved, but their threat situations would also be managed.

In terms of contribution to the realm of web application security, one of the key aspects that can be highlighted in connection with this project involves offering an effective and scalable method of ensuring protection against any illicit links for websites. Thanks to its design, SQLInsight allows easy integration with any existing web architecture and thus makes it feasible for a wide array of uses. At the same time, the knowledge gained from this investigation helped draw attention to the possibility of using machine learning approaches for improving cybersecurity practices via embedding the algorithms into the digital architecture.

As good as it might seem, SQLInsight has also proved the necessity of continuous evolution and progress in view of constantly emerging cyber threats. Such limitations as a lack of a larger dataset to train the model on, among others, suggest possible directions for further improvements.

7.1 Future Works

In this research paper, a model for detecting SQL attacks utilizing an LR algorithm has been developed. Future works may include the construction of a model that will allow us to identify not only SQL injection attacks but XSS attacks and PHP object injection attacks at once. The reason is that we will increase the effectiveness of the system by improving the security of the application, which will no longer only be protected from SQL attacks but will be secured against other types of attacks as well. Scaling up the system for SQLInsight to use it with larger web-based systems is needed in the future. Also, the improvement of the monitoring dashboard with advanced features such as predictive analysis and visualization will help enhance the performance of the system.

Another avenue that should be explored in future studies involves building an integration API for the firewall systems. The use of this API will allow the SQLInsight system to not only detect the SQL injections but also automatically undertake the necessary preventive steps of blocking the IP addresses responsible for the attack and disconnecting their services. Another advantage of integrating with the firewall will be automating the process and minimizing the chances of successful infiltration of the network system in question. Making this tool available via an API for others to use can also be considered in order to extend its usability.

Bibliography

[1] OWASP. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

[2] Kareem, F. Q., et al. (2021). SQL injection attacks prevention system technology: Review. *Asian Journal of Research in Computer Science*. https://doi.org/10.9734/ajrcos/2021/v10i330242

[3] ManageEngine. (n.d.). *SQL injection: What it is and how it works*. https://www.manageengine.com/log-management/cyber-security/sql-injection.html

[4] Chandrashekhar, R., Mardithaya, M., Thilagam, S., & Saha, D. (2012). SQL injection attack mechanisms and prevention techniques. *Lecture Notes in Computer Science*. https://doi.org/10.1007/978-3-642-29280-4_61

[5] Sadeghian, A., Zamani, M., & Abd. Manaf, A. (2014). SQL injection vulnerability general patch using header sanitization. *I4CT 2014 - 1st International Conference on Computer, Communications, and Control Technology*. https://doi.org/10.1109/I4CT.2014.6914182

[6] Goldberg, S. R., Kelleher, R. T., & Morse, W. H. (1975). Second order schedules of drug injection. *Federation Proceedings, 34*(9). https://doi.org/10.1007/978-1-4684-2634-2_4

[7] Singh, J. P. (2017). Analysis of SQL injection detection techniques. *Theoretical and Applied Informatics, 28*(1 & 2), 37–55. https://doi.org/10.20904/281-2037

[8] Abdullayev, V., & Chauhan, A. S. (2023). SQL injection attack: Quick view. *Mesopotamian Journal of CyberSecurity, 2023*. https://doi.org/10.58496/MJCS/2023/006

[9] Alaca, Y., Çelik, Y., & Goel, S. (2023). Anomaly detection in cyber security with graph-based LSTM in log analysis. *Chaos Theory and Applications*. https://doi.org/10.51537/chaos.1348302

[10] A. M. R., & V. P. (2022). Review of cyber attack detection: Honeypot system. *Webology, 19*(1). https://doi.org/10.14704/web/v19i1/web19370

[11] Gupta, V. K. (2024, September 2). *Honeypots 101: A beginner's guide to honeypots*. Medium. https://bevijaygupta.medium.com/honeypots-101-a-beginners-guide-to-honeypots-f05eab2b3d17

[12] Liu, H., & Lang, B. (2019). Machine learning and deep learning methods for intrusion detection systems: A survey. *Applied Sciences, 9*(20). https://doi.org/10.3390/app9204396

[13] Muneer, S., Farooq, U., Athar, A., Ahsan Raza, M., Ghazal, T. M., & Sakib, S. (2024). A critical review of artificial intelligence based approaches in intrusion detection: A comprehensive analysis. *Journal of Engineering, 2024*, 1–16. https://doi.org/10.1155/2024/3909173

[14] Abdallah, E. E., Eleisah, W., & Otoom, A. F. (2022). Intrusion detection systems using supervised machine learning techniques: A survey. *Procedia Computer Science*. https://doi.org/10.1016/j.procs.2022.03.029

[15] Mahesh, B. (2020). Machine learning algorithms: A review. *International Journal of Science and Research, 9*(1)

[16] Pasupa, K., & Sunhem, W. (2017). A comparison between shallow and deep architecture classifiers on small dataset. *Proceedings of 2016 8th International Conference on Information Technology and Electrical Engineering*, ICITEE 2016. https://doi.org/10.1109/ICITEED.2016.7863293

[17] Prinzi, F., Currieri, T., Gaglio, S., & Vitabile, S. (2024). Shallow and deep learning classifiers in medical image analysis. *European Radiology Experimental, 8*(1). https://doi.org/10.1186/s41747-024-00428-2

[18] Starbuck, C. (2023). *The fundamentals of people analytics*. Springer International Publishing. https://doi.org/10.1007/978-3-031-28674-2

[19] Kaur, N., & Himanshu. (2023). Logistic regression: A basic approach. *Lecture Notes in Networks and Systems*. https://doi.org/10.1007/978-981-19-9638-2_41

[20] Paluszek, M., & Thomas, S. (2020). What is deep learning? In *Practical MATLAB Deep Learning*. https://doi.org/10.1007/978-1-4842-5124-9_1

[21] Tang, P., Qiu, W., Huang, Z., Lian, H., & Liu, G. (2020). Detection of SQL injection based on artificial neural network. *Knowledge-Based Systems, 190*. https://doi.org/10.1016/j.knosys.2020.105528

[22] Fu, X., & Qian, K. (2008). SAFELI: SQL injection scanner using symbolic execution. *TAV-WEB 2008 - Proceedings of the Workshop on Testing, Analysis and Verification of Web Software*. https://doi.org/10.1145/1390832.1390838

[23] Rupasinghe, P. L., & Punyasiri, S. (n.d.). *Signature & behavior based malware detection*. https://doi.org/10.13140/RG.2.2.22127.20640

[24] Xiao, Z., Zhou, Z., Yang, W., & Deng, C. (2017). An approach for SQL injection detection based on behavior and response analysis. *2017 9th IEEE International Conference on Communication Software and Networks*, ICCSN 2017. https://doi.org/10.1109/ICCSN.2017.8230346

[25] Pattewar, T., Patil, H., Patil, H., Patil, N., Taneja, M., & Wadile, T. (2019). Detection of SQL injection using machine learning: A survey. *International Research Journal of Engineering and Technology, 6*(11).

[26] Moh, M., Pininti, S., Doddapaneni, S., & Moh, T. S. (2016). Detecting web attacks using multi-stage log analysis. *Proceedings - 6th International Advanced Computing Conference*, IACC 2016. https://doi.org/10.1109/IACC.2016.141

[27] Buehrer, G., Weide, B. W., & Sivilotti, P. A. G. (2005). Using parse tree validation to prevent SQL injection attacks. *SEM 2005 - Proceedings of the 5th International Workshop on Software Engineering and Middleware*. https://doi.org/10.1145/1108473.1108496

[28] Selvaganapathy, S. G., Nivaashini, M., & Natarajan, H. P. (2018). Deep belief network based detection and categorization of malicious URLs. *Information Security Journal, 27*(3). https://doi.org/10.1080/19393555.2018.1456577

[29] Potluri, S., Ahmed, S., & Diedrich, C. (2018). Convolutional neural networks for multi-class intrusion detection system. *Lecture Notes in Computer Science*. https://doi.org/10.1007/978-3-030-05918-7_20

[30] Tuor, A., Kaplan, S., Hutchinson, B., Nichols, N., & Robinson, S. (2017). Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. *AAAI Workshop - Technical Report*.

[31] Gambler Yu. (2024, May 29). *Biggest SQL injection dataset* [Dataset]. Kaggle. https://www.kaggle.com/datasets/gambleryu/biggest-sql-injection-dataset

[32] Saptajit. (2024, May 29). *SQL-injection-detection* [Dataset]. GitHub. https://github.com/saptajitbanerjee/SQL-Injection-Detection/blob/main/demo_good_and_bad_requests.csv

[33] Iniesta, M. (2024, May 29). *EDA - SQL injection dataset* [Dataset]. Kaggle. https://www.kaggle.com/code/iniestamoh/eda-sql-injection-dataset/input

Appendixes

Appendix A: ML Model Training Code

Below is the code written in Python 3 that trains the Machine Learning (ML) model for detecting the SQL injection attacks. This code has been run in Google Colab environment.d for training the model and to check how it will behave with the uploaded the dataset which contains more than 140 k samples which consist of malicious and normal queries.

Import necessary libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import drive
from sklearn.feature_selection import VarianceThreshold
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score,f1_score
from sklearn.metrics import confusion_matrix
import joblib
import shutil
from google.colab import files
```

Load Dataset

```
from google.colab import drive
drive.mount('/content/drive')
data = pd.read_csv('/content/drive/My Drive/SQLi/SQL_dataset.csv')
```

```
Mounted at /content/drive
```

Data cleaning and reprocessing

```
# Stage 1: Convert text into a collection of words
vectorizer = CountVectorizer(lowercase=False)
X = vectorizer.fit_transform(data['Query'])
# Stage 2: Remove frequently and infrequently used terms
sel = VarianceThreshold(threshold=(.0001 * (1 - .0001)))
X = sel.fit_transform(X)
# Stage 3: Convert every word into lowercase
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(data['Query'].str.lower())
y = data['Label']
# Stage 4: Remove missing rows
data.dropna(inplace=True)
```

Splitting Dataset

```
# Splitting the dataset into training and testing sets
X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Add text cell

Training & testing

```
# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Creating and Training the Model
model = LogisticRegression(max_iter=1000, solver='liblinear') # liblinear is more efficient for large
model.fit(X_train, y_train)
# Making predictions on the test set
y_pred = model.predict(X_test)
# Evaluating the Model
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
print(f"Precision: {precision*100:.2f}%")
print(f"Recall: {recall*100:.2f}%")
print(f"F1 Score: {f1*100:.2f}%")
print(classification_report(y_test, y_pred))
```

Confusion matrix

```
conf_matrix = confusion_matrix(y_test, y_pred)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=['Positive', 'Negative'], yticklabels=['Positive', 'Negative'])
plt.xlabel('Predictions')
plt.ylabel('Actual Values')
plt.title('Confusion Matrix')
plt.show()
```

Testing with New dataset

```
# Load the new dataset
new_data_path = '/content/drive/My Drive/SQLi/New_SQL_dataset.csv'
new_data = pd.read_csv(new_data_path)
# Preprocessing to the new dataset
new_X = vectorizer.transform(new_data['Query'])
new_y = new_data['Label']
# Predict using the trained model
new_predictions = model.predict(new_X)
# Evaluate the model on the new dataset
new_accuracy = accuracy_score(new_y, new_predictions)
new_precision = precision_score(new_y, new_predictions)
new_recall = recall_score(new_y, new_predictions)
new_f1 = f1_score(new_y, new_predictions)
print(f"New Data_Accuracy: {new_accuracy*100:.2f}%")
print(f"New Data_Precision: {new_precision*100:.2f}%")
print(f"New Data_Recall: {new_recall*100:.2f}%")
print(f"New Data_F1 Score: {new_f1*100:.2f}%")
# Confusion Matrix for the new dataset
new_conf_matrix = confusion_matrix(new_y, new_predictions)
sns.heatmap(new_conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=['Positive', 'Negative'], yticklabels=['Positive', 'Negative'])
plt.xlabel('Predictions')
plt.ylabel('Actual Values')
plt.title('Confusion Matrix for New Dataset')
plt.show()
```

Save & Export

```
# Save the model
joblib.dump(model, 'ML_model.pkl')
# Define path
colab_model_path = 'ML_model.pkl'
# Download the model file
files.download(colab_model_path)
# Save the vectorizer
joblib.dump(vectorizer, 'vectorizer-2.joblib')
# Define path
colab_vectorizer_path = 'vectorizer-2.joblib'
# Download the vectorizer file
files.download(colab_vectorizer_path)
```

Appendix B: Server-side Programming Code

The server-side code that was used for deploying the trained Machine Learning model for detecting SQL injections attacks is presented in this appendix. This code has been developed using Python programming language version 3 and uses several libraries for performing various operations.

Access logs

```
from __future__ import annotations

import re
import sys
from datetime import datetime, timezone
from pathlib import Path
from urllib.parse import unquote

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from config import ACCESS_LOG_PATH  # noqa: E402

_IP_RE = re.compile(r"^(\d{1,3}(?:\.\d{1,3}){3})")
# Thesis-faithful: capture the request target between method and HTTP
version.
_REQ_RE = re.compile(r'"(GET|POST|PUT|DELETE|HEAD)\s(\S+)\sHTTP')


def write_access_log(ip: str, method: str, path: str, status: int = 200,
                     size: int = 512, user_agent: str = "-", referer: str =
"-") -> str:
    ts = datetime.now(timezone.utc).strftime("%d/%b/%Y:%H:%M:%S +0000")
    line = f'{ip} - - [{ts}] "{method} {path} HTTP/1.1" {status} {size}
"{referer}" "{user_agent}"\n'
    try:
        ACCESS_LOG_PATH.parent.mkdir(parents=True, exist_ok=True)
        with open(ACCESS_LOG_PATH, "a", encoding="utf-8") as f:
            f.write(line)
    except OSError as e:
        print(f"[accesslog] could not write log: {e}")
    return line


def extract_ip(log_line: str) -> str | None:
    m = _IP_RE.search(log_line)
    return m.group(1) if m else None


def extract_query(log_line: str) -> str:
    """Extract the query string value from an access-log line.

    Returns the decoded value after the first '=' of the query string, e.g.
    `GET /search?q=<payload> HTTP/1.1` -> `<payload>`. Faithful to the
thesis.
    """
    m = _REQ_RE.search(log_line)
    if not m:
        return ""
    target = m.group(2)
    if "?" not in target:
        return ""
    qs = target.split("?", 1)[1]
```

```
    # Take the value of the first parameter (e.g. q=...).
    first = qs.split("&", 1)[0]
    value = first.split("=", 1)[1] if "=" in first else first
    return unquote(value.replace("+", " "))


def extract_user_agent(log_line: str) -> str | None:
    parts = log_line.split('"')
    return parts[-2] if len(parts) >= 6 else None
```

alerts

```
from __future__ import annotations

import smtplib
import sys
import time
from email.message import EmailMessage
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from config import (  # noqa: E402
    ALERT_COOLDOWN_SECONDS,
    ALERT_FROM,
    ALERT_TO,
    ALERTS_ENABLED,
    SMTP_HOST,
    SMTP_PASSWORD,
    SMTP_PORT,
    SMTP_USER,
)

_last_sent: dict[str, float] = {}


def _cooling_down(ip: str | None) -> bool:
    key = ip or "_"
    now = time.time()
    if now - _last_sent.get(key, 0) < ALERT_COOLDOWN_SECONDS:
        return True
    _last_sent[key] = now
    return False


def _html_body(event: dict) -> str:
    geo = ", ".join(
        x for x in [event.get("city"), event.get("region"),
event.get("country")] if x
    ) or "Unknown"
    return f"""\
<div style="font-family:Arial,sans-serif;max-width:600px;border:1px solid
#eee;border-radius:10px;overflow:hidden">
  <div style="background:#7c3aed;color:#fff;padding:16px 20px">
    <h2 style="margin:0">&#9888; SQLInsight Security Alert</h2>
    <p style="margin:4px 0 0;opacity:.9">SQL Injection attempt detected</p>
  </div>
  <table style="width:100%;border-collapse:collapse;font-size:14px">
    <tr><td style="padding:10px 20px;color:#666">Date / Time (UTC)</td><td
style="padding:10px 20px"><b>{event.get('ts','')}</b></td></tr>
```

```
    <tr style="background:#fafafa"><td style="padding:10px
20px;color:#666">Source IP</td><td style="padding:10px
20px"><b>{event.get('source_ip','?')}</b></td></tr>
    <tr><td style="padding:10px 20px;color:#666">Location</td><td
style="padding:10px 20px">{geo}</td></tr>
    <tr style="background:#fafafa"><td style="padding:10px
20px;color:#666">Attack Type</td><td style="padding:10px
20px">{event.get('attack_type') or 'Unknown'}</td></tr>
    <tr><td style="padding:10px 20px;color:#666">Confidence</td><td
style="padding:10px
20px">{round(float(event.get('confidence',0))*100,1)}%</td></tr>
    <tr style="background:#fafafa"><td style="padding:10px
20px;color:#666">Status</td><td style="padding:10px 20px"><b
style="color:#dc2626">SUSPICIOUS</b></td></tr>
    <tr><td style="padding:10px 20px;color:#666;vertical-align:top">Payload
/ Log</td>
        <td style="padding:10px 20px"><code style="word-break:break-
all">{(event.get('log_line') or
event.get('query',''))[:500]}</code></td></tr>
  </table>
  <div style="padding:12px 20px;background:#f3f4f6;color:#888;font-
size:12px">Sent automatically by SQLInsight IDS.</div>
</div>"""


def send_alert(event: dict) -> bool:
    """Send an alert email. Returns True if a message was actually sent."""
    if not ALERTS_ENABLED:
        return False
    if not (SMTP_USER and SMTP_PASSWORD and ALERT_TO):
        print("[alerts] SMTP not configured; skipping email.")
        return False
    if _cooling_down(event.get("source_ip")):
        return False

    message = EmailMessage()
    message["From"] = ALERT_FROM
    message["To"] = ALERT_TO
    message["Subject"] = "Security Alert: SQL Injection Attempt Detected"
    message.set_content(
        f"SQL Injection detected from {event.get('source_ip','?')} at
{event.get('ts','')}.\n"
        f"Attack type: {event.get('attack_type') or 'Unknown'}\n"
        f"Payload: {(event.get('query') or '')[:500]}"
    )
    message.add_alternative(_html_body(event), subtype="html")

    try:
        with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=15) as server:
            server.starttls()
            server.login(SMTP_USER, SMTP_PASSWORD)
            server.send_message(message)
        print(f"[alerts] Alert email sent to {ALERT_TO}.")
        return True
    except Exception as e:  # noqa: BLE001 - alerting must never crash
detection
        print(f"[alerts] Error sending email: {e}")
        return False
```

frontend

```
from __future__ import annotations

import json
import os
import sys
from pathlib import Path

from flask import Flask, jsonify, request, send_from_directory

_BACKEND_DIR = Path(__file__).resolve().parent
sys.path.insert(0, str(_BACKEND_DIR))
sys.path.insert(0, str(_BACKEND_DIR.parent))

import accesslog  # noqa: E402
import alerts  # noqa: E402
import database  # noqa: E402
import detection  # noqa: E402
import geoip  # noqa: E402
from config import (  # noqa: E402
    ALERTS_ENABLED,
    FRONTEND_DIST,
    HOST,
    METRICS_PATH,
    PORT,
    VERSION,
)

app = Flask(__name__, static_folder=None)
database.init_db()

DEFAULT_UA = "SQLInsight-Client"


@app.get("/api/health")
def health():
    return jsonify(
        status="ok",
        version=VERSION,
        model_loaded=detection.model_loaded(),
        alerts_enabled=ALERTS_ENABLED,
    )


@app.get("/api/model")
def model_info():
    if METRICS_PATH.exists():
        return jsonify(json.loads(METRICS_PATH.read_text()))
    return jsonify(error="metrics not found; run ml/train_model.py"), 404


@app.post("/api/scan")
def scan():
    data = request.get_json(silent=True) or {}
    query = (data.get("query") or "").strip()
    if not query:
        return jsonify(error="missing 'query'"), 400

    source_ip = data.get("source_ip") or request.remote_addr or "127.0.0.1"
    source = data.get("source") or "scanner"
```

```
    method = (data.get("method") or "GET").upper()
    user_agent = data.get("user_agent") or request.headers.get("User-
Agent", DEFAULT_UA)
    path = data.get("path") or "/search"

    result = detection.detect(query)
    geo = geoip.geolocate(source_ip)

    # Write an Apache-style access log line (feeds the standalone monitor
too).
    from urllib.parse import quote
    log_line = accesslog.write_access_log(
        source_ip, method, f"{path}?q={quote(query)}",
user_agent=user_agent
    )

    event = {
        **result,
        "source_ip": source_ip,
        "method": method,
        "path": path,
        "user_agent": user_agent,
        "source": source,
        "log_line": log_line.strip(),
        **geo,
        "alerted": 0,
    }

    if result["prediction"] == 1:
        if alerts.send_alert(event):
            event["alerted"] = 1

    event_id = database.insert_event(event)
    event["id"] = event_id
    return jsonify(event)


@app.post("/api/reset")
def reset():
    """Clear all recorded detections (resets the dashboard counters to
zero)."""
    cleared = database.clear_events()
    return jsonify(status="ok", cleared=cleared)


@app.get("/api/events")
def events():
    limit = min(int(request.args.get("limit", 50)), 500)
    verdict = request.args.get("verdict")
    return jsonify(events=database.recent_events(limit=limit,
verdict=verdict))


@app.get("/api/stats")
def stats():
    data = database.get_stats()
    if METRICS_PATH.exists():
        m = json.loads(METRICS_PATH.read_text())
        data["model"] = m.get("headline", {})
        data["model"]["generalisation"] = m.get("experiment_2_unseen", {})
    return jsonify(data)
```

```
# ---- Serve the pre-built React dashboard (frontend/dist) ----
@app.get("/")
@app.get("/<path:resource>")
def spa(resource: str = ""):
    if resource.startswith("api/"):
        return jsonify(error="not found"), 404
    if not FRONTEND_DIST.exists():
        return (
            "<h1>SQLInsight backend is running</h1>"
            "<p>Frontend not built yet. Run <code>cd frontend &amp;&amp;
npm install &amp;&amp; npm run build</code>, "
            "or use the API at <code>/api/health</code>.</p>",
            200,
        )
    target = FRONTEND_DIST / resource
    if resource and target.exists() and target.is_file():
        return send_from_directory(FRONTEND_DIST, resource)
    return send_from_directory(FRONTEND_DIST, "index.html")


def main() -> None:
    print(f"SQLInsight v{VERSION}  |
model_loaded={detection.model_loaded()}  alerts={ALERTS_ENABLED}")
    print(f"Dashboard: http://{HOST}:{PORT}")
    app.run(host=HOST, port=PORT, debug=bool(os.getenv("FLASK_DEBUG")),
threaded=True)


if __name__ == "__main__":
    main()
```

Database backend

```
from __future__ import annotations

import sqlite3
import sys
import threading
from datetime import datetime, timedelta, timezone
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from config import DB_PATH  # noqa: E402

_local = threading.local()

_SCHEMA = """
CREATE TABLE IF NOT EXISTS events (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    ts          TEXT    NOT NULL,
    query       TEXT    NOT NULL,
    prediction  INTEGER NOT NULL,
    confidence  REAL    NOT NULL,
    verdict     TEXT    NOT NULL,
    attack_type TEXT,
    source_ip   TEXT,
    method      TEXT,
    path        TEXT,
```

```
    user_agent  TEXT,
    country     TEXT,
    region      TEXT,
    city        TEXT,
    lat         REAL,
    lon         REAL,
    source      TEXT,
    alerted     INTEGER DEFAULT 0
);
CREATE INDEX IF NOT EXISTS idx_events_ts ON events(ts);
CREATE INDEX IF NOT EXISTS idx_events_pred ON events(prediction);
"""


def _conn() -> sqlite3.Connection:
    if getattr(_local, "conn", None) is None:
        _local.conn = sqlite3.connect(DB_PATH, check_same_thread=False)
        _local.conn.row_factory = sqlite3.Row
        _local.conn.executescript(_SCHEMA)
    return _local.conn
```

Detection

```
from __future__ import annotations

import re
import sys
from functools import lru_cache
from pathlib import Path

import joblib

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from config import MODEL_PATH, VECTORIZER_PATH  # noqa: E402

# Verdict threshold on P(malicious). 0.5 is the natural LR boundary and was
# validated to catch all canonical attacks while keeping precision ~98.5%.
THRESHOLD = 0.5

_ATTACK_PATTERNS: list[tuple[str, re.Pattern]] = [
    ("Tautology / Auth Bypass",
re.compile(r"(\bor\b|\band\b)\s+['\"]?\d+['\"]?\s*=\s*['\"]?\d+", re.I)),
    ("Tautology / Auth Bypass",
re.compile(r"['\"]\s*(or|and)\s*['\"]?\w+['\"]?\s*=\s*['\"]?\w+", re.I)),
    ("Union-based", re.compile(r"\bunion\b(\s+all)?\s+select\b", re.I)),
    ("Stacked Queries",
re.compile(r";\s*(drop|insert|update|delete|create|alter|exec)\b", re.I)),
    ("Time-based Blind",
re.compile(r"\b(sleep|pg_sleep|waitfor\s+delay|benchmark)\b", re.I)),
    ("Error-based",
re.compile(r"\b(extractvalue|updatexml|utl_inaddr|convert\s*\(|cast\s*\()",
re.I)),
    ("Command Execution",
re.compile(r"\b(xp_cmdshell|exec\s*\(|sp_executesql)\b", re.I)),
    ("Comment Injection", re.compile(r"(--|#|/\*)", re.I)),
    ("Schema Enumeration",
re.compile(r"\b(information_schema|sys\.|all_tables|table_name|version\s*\(
)", re.I)),
]
```

```
@lru_cache(maxsize=1)
def _load():
    if not MODEL_PATH.exists() or not VECTORIZER_PATH.exists():
        raise FileNotFoundError(
            f"Model artifacts not found. Run `python ml/train_model.py`
first.\n"
            f"  expected: {MODEL_PATH}\n            {VECTORIZER_PATH}"
        )
    model = joblib.load(MODEL_PATH)
    vectorizer = joblib.load(VECTORIZER_PATH)
    return model, vectorizer


def model_loaded() -> bool:
    try:
        _load()
        return True
    except Exception:
        return False


def classify_attack_type(query: str) -> str:
    for label, pattern in _ATTACK_PATTERNS:
        if pattern.search(query):
            return label
    return "Generic / Obfuscated"


def detect(query: str) -> dict:
    """Classify a single query/payload.

    Returns: {query, prediction(0|1), confidence(float), verdict,
attack_type}
    """
    query = "" if query is None else str(query)
    model, vectorizer = _load()
    X = vectorizer.transform([query])
    proba = float(model.predict_proba(X)[0][1])
    prediction = int(proba >= THRESHOLD)
    return {
        "query": query,
        "prediction": prediction,
        "confidence": round(proba, 4),
        "verdict": "Suspicious" if prediction == 1 else "Normal",
        "attack_type": classify_attack_type(query) if prediction == 1 else
None,
    }
```